

TEMA 3

- Estructuras de control
- Estructuras de selección
- Estructuras de repetición
- Estructuras de salto
- Subrutinas
- Paso de parámetros
- Ejecución

Usando las sentencias if y switch

if-else

El formato básico de la sentencia **if** es el siguiente:

```
if (Expresión Booleana) {  
    System.out.println("Dentro de la sentencia condicional.")  
}
```

La sentencia **if-else**:

```
if (x>3) {  
    System.out.println("x es mayor que 3");  
}else{  
    System.out.println("x es menor que 3");  
}
```

Las llaves son opcionales si tenemos sólo una sentencia para ejecutar dentro del cuerpo de la sentencia condicional. Esta práctica no es muy recomendable.

Podemos anidar una sentencia condicional dentro de otra:

```
if(precio <300){  
    comprarProducto( );  
}else{  
    if(precio <400){  
        pedirAprobación( );  
    }else{  
        recazarProducto( );  
    }  
}
```

Aunque si podemos debemos evitarlo para no perder legibilidad. El código anterior se podría escribir:

```
if(precio <300){
```

```

    comprarProducto( );
}else if(precio <400){
    pedirAprobación( );
}else{
    recazarProducto( );
}

```

Unas pocas reglas para usar la sentencia condicional:

- Podemos tener **cero o un else** para un if dado y debe aparecer al final, después de los else if.
- Podemos tener de **cero a varios else if** para un if dado, y deben de aparecer antes del else.
- Una vez que la condición de un else if se cumple, el código restante será comprobado (ni los restantes else if ni el else).

```

int x=1;
if (x==3) { }
else if (x<4) { System.out.println("<4"); }
else if (x<2) { System.out.println("<2"); }
else { System.out.println("else"); }

```

Expresiones legales para sentencias if

La expresión en una sentencia if debe ser una **expresión booleana**. Cualquier expresión que se resuelva a **true** o **false** es válida.

```

int y=5;
int x=2;
if(((x>3) && (y<2)) | doStuff( )){
    System.out.println("true");
}

```

Sentencias switch

La sentencia switch proporciona una manera más clara de manejar lógicas de decisión complejas. Por lo demás no hace nada que no se pueda hacer con una sentencia condicional normal. Estos dos bloques son equivalentes:

```

int x = 3;
if ( x == 1) {
    System.out.println(" x vale 1");
}
else if ( x == 2) {
    System.out.println(" x vale 2");
}

```

```

int x=3;
switch(x) {
    case 1:
        System.out.println("x vale 1");
        break;
    case 2:
        System.out.println("x vale 1");
}

```

```

else{                                     break;
    System.out.println("¿Qué es x? ");    default:
}                                           System.out.println("¿Qué es x? ");
                                           }

```

La forma general de una sentencia switch es:

```

switch (expresión) {
    case constante1: bloque de código
    case constante2: bloque de código
    .....
    default: bloque de código
}

```

En la expresión el switch puede evaluar **char, byte, short, int, enum y String**. Esto significa que si no estamos usando una **enumeración** o un **String**, sólo las variables y valores que pueden ser promovidas (**cast implícito**) a un **int** son aceptables. No compilará nada más, incluyendo el resto de valores numéricos **long, float o double**.

Una **constante case** se debe de evaluar al mismo tipo que el que la expresión switch puede usar, con una gran restricción: la constante case debe ser constante en el tiempo de compilación. Como el argumento case tiene que ser resuelto en tiempo de compilación, sólo podemos utilizar una constante o una variable final que sean **inmediatamente inicializadas** con un valor literal. No es suficiente con que sea final; **debe ser una constante en tiempo de compilación**.

```

final int a = 1;
final int b;
b = 2;
int x = 0;
switch (x) {
    case a: // ok
    case b: // error de compilación

```

El switch solo puede comprobar la igualdad (no relaciones mayor que, menor que,...). Podemos usar métodos, pero deben devolver un valor compatible con int.

```

String s = "xyz";
switch (s.length()) {
    case 1:
        System.out.println("La longitud es 1");
        break;
    case 2:
        System.out.println("La longitud es 2");
        break;

```

```

case 3:
    System.out.println("La longitud es 3");
    break;
default:
    System.out.println("No coinciden.");
}

```

¿Qué ocurre si hacemos switch en una variable más pequeña que int?

```

byte g = 2;
switch(g) {
    case 23:
    case 128: // Argumento demasiado largo para un byte.
}

```

También es ilegal usar dos o más veces el mismo valor en los case. Es legal usar el poder del **boxing**:

```

switch ( new Integer(4) ){ case 4: System.out.println("Me gusta el boxing"); }

```

Break y el flujo en bloques switch

Las constantes case son evaluadas de arriba hacia abajo, y la primera coincidencia con la expresión del switch es el punto de entrada de ejecución. Se ejecutará el bloque de código asociado y todos los bloques que le sigan (de los restantes case que estén por debajo) también. A no ser que aparezca la palabra **break**.

```

class SwitchString {
    public static void main(String[ ] args){
        String s = "verde";
        switch(s){
            case "rojo": System.out.println("rojo");
            case "verde": System.out.println("verde");
            case "azul": System.out.println("azul");
            default : System.out.println("Hecho");
        }
    }
}

```

Cuando el programa encuentra la palabra **break** durante la ejecución de un sentencia switch, la ejecución inmediatamente sale del bloque switch a la siguiente sentencia después de este.

```

class SwitchString {
    public static void main(String[ ] args){
        String s = "verde";
        switch(s){

```

```

    case "rojo": System.out.println("rojo"); break;
    case "verde": System.out.println("verde"); break;
    case "azul": System.out.println("azul"); break;
    default : System.out.println("Hecho");
}
}
}

```

El case **default** se ejecuta cuando no hay ninguna coincidencia con los case anteriores (es equivalente al caso **else** del bloque condicional **if**). Puede aparecer o no, pero en caso de aparecer es aconsejable que aparezca al final. Sin embargo esto no es obligatorio.

```

int x=7;
switch (x)
{
    case 2: System.out.println("2");
    default: system.out.println("default");
    case 3: System.out.println("3");
    case 4: System.out.println("4");
}

```

Creando bucles

Usando el bucle while

El bucle **while** es apropiado cuando no sabemos cuántas veces se tiene que repetir la ejecución de un bloque de código, pero queremos que se repita mientras se cumpla una condición, es decir, **mientras la condición sea true**.

```

while ( expresión ) {
    // Código aquí.
}

```

```

int x = 2;
while ( x == 2 ) {
    System.out.println(x);
    ++x;
}
System.out.println("Se ha pasado el bucle.");

```

Usando bucles do-while

El bucle do-while es similar al bucle while, excepto por que la expresión no es evaluada hasta que el código es ejecutado. En consecuencia, **está garantizado que el código dentro del bucle do-while se ejecuta por lo menos una vez** (independientemente de que se cumpla o no la condición de la expresión).

```
do{
    System.out.println("Dentro del bucle");
}while(false);
```

Usando el bucle for

- El bucle for básico

El bucle **for** es especialmente útil para controlar el flujo cuando ya **sabemos cuántas veces necesitamos ejecutar las sentencias** del bloque del bucle. La declaración del bucle **for** tiene tres partes principales, además del cuerpo del bucle: La declaración e inicialización de variables, la expresión booleana (condición) y la expresión de iteración.

```
for ( /*Inicialización*/ ; /*Condición*/ ; /*Iteración*/ ) {
    /* Cuerpo del bucle */
}
```

```
for ( int i= 0 ; i<10 ; i++ )
{
    System.out.println("i es: " +i );
}
```

- Declaración e inicialización

La primera parte de la sentencia **for** permite declarar e inicializar cero, una o múltiples variables del mismo tipo dentro de los paréntesis después de la palabra clave **for**. Si declaramos más de una variable al mismo tiempo, necesitaremos separarlas con comas:

```
for ( int x = 10, y = 3; y > 3; y++ ) { }
```

Las variables declaradas son locales al bucle y no pueden ser usadas fuera de este.

- Expresión condicional (boolean)

La expresión condicional se evalúa a un valor booleano. Se puede asignar cualquier expresión lógica compleja, **pero solamente una**.

```
for ( int x=0; ((( x < 10 ) && ( y-- > 2 )) | x==3)); x++ ) { }
```

- Expresión de iteración

Después de cada ejecución del cuerpo del bucle `for`, la expresión de iteración es ejecutada. Aquí decidimos que queremos que ocurra con cada iteración del bucle. Hay que tener en cuenta que excepto en el caso de una salida forzada, **la evaluación de la expresión de iteración y la posterior evaluación de la expresión condicional son siempre las dos últimas cosas que suceden en un bucle `for`.**

```
static boolean hacerCosas()
for ( int x =0 ; x<3 ; x++ ) {
    System.out.println("en el bucle for");
    return true;
}
return true;
} // ¿Qué ocurre?
```

El **`return`** causa que se abandone, no sólo la iteración en curso, **sino el método entero**. Las causas de terminación temprana de un bucle son:

Code in Loop	What Happens
<code>break</code>	Execution jumps immediately to the first statement after the <code>for</code> loop.
<code>return</code>	Execution jumps immediately back to the calling method.
<code>System.exit()</code>	All program execution stops; the VM shuts down.

- **Otras consideraciones**

Ninguna de las tres secciones anteriores es obligatoria. El siguiente ejemplo es legal, aunque en la práctica causaría un bucle infinito:

```
for( ; ; ){
    System.out.println("Dentro del bucle infinito");
}
```

En ausencia de las secciones de inicialización e incremento el bucle actúa como un bucle `while`:

```
int i = 0;
for( ; i<10 ; ) {
    i++;
}
```

Otros ejemplos complejos:

```
for( int i = 0, j = 0; (i<10) && (j<10); i++ , j++ ){
    System.out.println(" i es: " + i + "y j es: " + j );
}
```

```
}
```

```
int b=3;  
for ( int a = 1; b != 1; System.out.println("iterate")) {  
    b = b - a;  
}
```

- **El bucle for mejorado (para Arrays)**

El bucle for mejorado es un for especializado que simplifica la iteración a través de un **array** o de una **collection**. En lugar de tener tres componentes, el for mejorado tiene dos:

```
int [ ] a = { 1,2,3,4 };  
for ( int x = 0; x <a.length; x++ )  
    System.out.print(a[x]);  
for( int n : a )  
    System.out.print(n);
```

for (declaración : expresión)

- **Declaración:** La variable de nueva declaración de un tipo compatible con los elementos del array al que estamos accediendo. Esta variable estará disponible dentro del bloque for, y su valor será el mismo que el elemento de array actual.
- **Expresión:** Es el array sobre el que queremos iterar: Puede ser directamente una variable array o una invocación a un método que devuelva un array. El array puede ser de cualquier tipo: primitivas, objetos o incluso arrays de arrays.

Usando break y continue

Las palabras clave **break** y **continue** son utilizadas para para el bucle completo (break) o sólo la iteración actual (continue). La sentencia **continue** debe ir dentro del bucle, de otra forma obtendremos un error de compilación. Las sentencias break deben ser usadas dentro de un bucle o un switch.

- La **sentencia break** causa que el programa pare la ejecución del bucle y comience a procesar la siguiente línea del código después del bucle.
- La **sentencia continue** cause que sólo la iteración del bucle cese y comience la siguiente iteración. Simplemente aborta la iteración actual y va a la siguiente.

```
for ( int i = 0; i < 10 ; i++ ){  
    if (i == 5){ continue; }
```



```
System.out.println("Dentro del bucle ");  
}
```

Sentencias sin etiquetar (unlabeled) y etiquetadas (labeled)

Aunque muchas sentencias en Java pueden tener etiquetas, lo más común es usar etiquetas con bucles como **for** o **while**, en conjunción con las sentencias **break** y **continue**.

Una sentencia etiqueta debe ser colocada justa antes de la sentencia que está siendo etiquetada, y consiste en un identificador válido que termina en dos puntos (:).

Es importante conocer la diferencia entre **break** y **continue** etiquetados o sin etiquetar. Las variedades etiquetadas son necesarias sólo en situaciones en las que tenemos un bucle anidado, y deseamos indicar a cuál de los bucles anidados queremos mandar el flujo, o desde cuál de los bucles anidados queremos que continúe la siguiente iteración. Las etiquetas deben de seguir las reglas para nombres de variables válidos.

```
boolean isTrue = true;  
outer:  
for( int i=0; i<5; i++ ){  
    while (isTrue) {  
        System.out.println("Hola");  
        break outer;  
    }  
    System.out.println("bucle externo.");  
}  
System.out.println("Adios");
```

En este ejemplo la palabra "Hola" se imprimirá una vez. Entonces el break etiquetado será ejecutado, y el flujo saldrá fuera del bucle etiquetado outer. La siguiente línea imprimirá "Adios".

```
outer:  
for( int i=0; i<5; i++ ){  
    for( int j=0; j<5; j++ ){  
        System.out.println("Hola");  
        continue outer;  
    }  
    System.out.println("bucle externo.");  
}  
System.out.println("Adios");
```

En este ejemplo, "Hola" será impreso cinco veces. Después de que la sentencia continue sea ejecutada, el flujo continua con la siguiente iteración del bucle

identificado con la etiqueta. Finalmente, cuando la condición en bucle externo se evalúa a false, este bucle terminará y se imprimirá “Adios”.